

```

Workers Launched: 2
Buffers: shared hit=487 read=7264
-> Hash Join (cost=8.30..12803.38 rows=1 width=0) (actual
time=44.493..134.889 rows=7 loops=3)
    Inner Unique: true
    Hash Cond: (p.author = u.pk)
    Buffers: shared hit=487 read=7264
    Worker 0:  actual time=132.750..132.751 rows=0 loops=1
                Buffers: shared hit=238 read=3192
    Worker 1:  actual time=0.667..134.246 rows=4 loops=1
                Buffers: shared hit=162 read=2120
    -> Parallel Seq Scan on forum.posts p (cost=0.00..12792.33
rows=1042 width=4) (actual time=0.132..133.575 rows=6667 loops=3)
        Output: p.pk, p.title, p.content, p.author, p.category,
p.reply_to, p.created_on, p.last_edited_on, p.editable, p.likes

;
    Finalize Aggregate (cost=114848.33..114848.34 rows=1 width=8)
(actual time=5190.322..5190.323 rows=1 loops=1)
    -> Gather (cost=114848.12..114848.33 rows=2 width=8) (actual
time=5189.678..5193.226 rows=3 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        -> Partial Aggregate (cost=113848.12..113848.13 rows=1
width=8) (actual time=4861.705..4861.712 rows=1 loops=3)
            -> Hash Join (cost=8.30..113848.09 rows=10
width=0) (actual time=2477.949..4861.639 rows=27 loops=3)
            ...

```

As you can see, the output now includes the same information that `EXPLAIN ANALYZE` would report.

Summary

PostgreSQL has a very complex cost-based query planner and optimizer that does its best to provide the fastest access to the underlying data.

Thanks to the EXPLAIN command, database administrators can monitor queries to track down the costs and the time taken for execution, and decide on how to improve them in order to get faster results. Usually, the creation of indexes is the less intrusive choice in query tuning, and PostgreSQL has a very rich and expressive index interface that allows the creation of single-column, multi-column, and partial indexes of different types and technologies. When indexes do not suffice, query rewriting could be a possible solution to perform query tuning.

Costs used by the planner are based on statistical data that has to be kept, as much as possible, up to date. While the auto-analyze daemon aims to do this, the DBA can always rely on the manual ANALYZE command to update the statistics.

Understanding a query plan, knowing which nodes are involved and what they imply on the query execution, understanding when the statistics are out of date, and being able to experiment with different query access methods are complex tasks that every DBA should learn.

We also explored the auto-explain extension, which can be used to automate the collection of information about plans chosen by the optimizer so that the DBA can easily inspect which queries are running poorly.

It is important to emphasize that performance tuning is one of the most complex tasks in database administration and there is no silver bullet or one-size-fits-all solution, so experience and a lot of practice are required. In the next chapter, we will start to gain that experience by trying our hand at logging and auditing.

Verify your knowledge

- How can you inspect the plan of a query?

The special command EXPLAIN allows you to inspect how PostgreSQL is going to execute a given query, showing a “node” for each execution step. See the *The EXPLAIN statement* section for more details.

- What is the difference between EXPLAIN and EXPLAIN EXPLAIN?

The EXPLAIN command will not execute the query, computing only the access plan; on the other hand, the EXPLAIN ANALYZE command will execute the query and print the query plan in the output. See the *EXPLAIN ANALYZE* section for more details.

- How does PostgreSQL keep the statistics up to date?

The statistics are updated every time a manual `ANALYZE` command is executed or the auto-vacuum (auto-analyze) daemon runs against a table. See the *ANALYZE and how to update statistics* section for more details.

- How does PostgreSQL choose to use a specific access method (e.g., an index)?

The optimizer decides the path to the data depending on the cost of each access method: the method with the lowest cost wins and is used to access the underlying data. See the *The optimizer* section for more details.

- What is the `auto_explain` extension?

The `auto_explain` extension allows the system to automatically output the query execution plan to the logs whenever the query reaches a defined threshold (e.g., the execution time exceeds a predefined limit). This allows the DBA to automatically get information about problematic queries. See the *Auto-explain* section for more details.

References

- PostgreSQL official documentation about `CREATE INDEX`: <https://www.postgresql.org/docs/current/sql-createindex.html>
- PostgreSQL official documentation about `pg_stats`: <https://www.postgresql.org/docs/current/view-pg-stats.html>
- PostgreSQL official documentation about `EXPLAIN`: <https://www.postgresql.org/docs/current/using-explain.html>
- PostgreSQL official documentation about `ANALYZE`: <https://www.postgresql.org/docs/current/sql-analyze.html>
- Auto-explain official documentation: <https://www.postgresql.org/docs/current/auto-explain.html>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



14

Logging and Auditing

PostgreSQL provides a very rich logging infrastructure. Being able to examine logs is a key skill for every database administrator—logs provide hints and information about what the cluster has done, what it is doing, and what happened in the past. This chapter will explain the basics of PostgreSQL log configuration, providing you with an explanation of how to configure the logging machinery to get the information you need about cluster activity. Logs can be analyzed manually, but database administrators often also exploit automated tools that can provide a wider insight into the cluster activity. Related to logging is the topic of auditing, which is the capability of tracking who did what to which data. Auditing is often enforced by government laws, rather than the needs of the database administrators. However, a good auditing system can also help administrators to identify what happened in the database.

In this chapter, you will learn about the following topics:

- Introduction to logging
- Extracting information from logs using pgBadger
- Implementing auditing

Technical requirements

You will need to know the following:

- How to manage PostgreSQL configurations
- How to start, restart, and monitor PostgreSQL and interact with PGDATA files

The chapter examples can be run on the `chapter_14`, Docker image, which you can find in the book's GitHub repository: <https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition>. For installation and usage of the Docker images available for this book, please refer to the instructions in *Chapter 1, Introduction to PostgreSQL*.

Introduction to logging

Like many other services and databases, PostgreSQL provides its own logging infrastructure so that the administrator can always inspect what the daemon processes are doing and what the current status of the database system is. While logs are not vital for the data and database activities, they represent very important knowledge about what has happened or is happening in the whole system, and they provide an important clue by means of which an administrator can take action.

PostgreSQL has a very flexible and configurable log infrastructure that allows different logging configuration, rotation, archiving, and post-analysis.

Logs are stored in a textual form, so that they can be easily analyzed with common log analysis tools, including operating system utilities such as `grep(1)`, `sed(1)`, and text editors.



The term “log,” as used in this chapter, refers only to the system’s textual logs, and not to the **Write-Ahead Logs (WALs)** that, on the other hand, are crucial in the database life cycle (see *Chapter 11, Transactions, MVCC, WALs, and Checkpoints*).

In a default installation, logs are contained in a specific sub-folder of the `PGDATA` directory, but as you will see in the following subsections, you are free to move logs to pretty much wherever you want in your operating system storage.

Every event that happens in the database is logged in a separate line of text within the logs, an important and useful aspect when you want to analyze logs with line-oriented tools such as the common Unix commands (for example, `grep(1)`). Of course, writing a huge amount of information into logs has drawbacks; it requires system resources and can fill the storage where the logs are placed. For this reason, it is important to manage the logging infrastructure according to the aim of the cluster, therefore logging only the minimum amount of information that can be used for post-analysis.



Logs can quickly fill your disk storage if you don’t configure them appropriately, and therefore you should be sure your cluster is not producing more logs than your system can handle.

Following the common Unix philosophy, PostgreSQL allows you to send logs to an external component named the syslog. The idea is that there could be, in your own infrastructure, a component or a machine that is responsible for collecting logs from all the available services, including databases, web servers, application servers, and so on. Therefore, you can redirect PostgreSQL logs to the same common syslog facility and get the cluster logs collected in the very same place as you already do for the other services. However, this is not always a good choice, and it is for this reason that PostgreSQL provides its own component, named the *logging collector*, to store logs.

In fact, under a heavy load, the syslog centralized collector could start to discard (and therefore lose) log entries, while the PostgreSQL logging collector has been designed explicitly to not lose a single piece of log information. Therefore, the logging collector shipped with PostgreSQL is usually the preferred way of keeping track of logs, so that you can be sure that once you start to analyze the logs, you have all the information the cluster has produced, with nothing missing.

PostgreSQL logging is configured via tunables contained in the main cluster configuration, namely the `postgresql.conf` file. In the following subsections, you will be introduced to the PostgreSQL logging configuration, and you will see how to tune your own log to match your needs.

Where to log

The first step in configuring the logging system is to decide where and how to store textual logs. The main parameter that controls the logging system is `log_destination`, which can assume one or more of the following values:

- `stderr` means the cluster logs will be sent to the standard error of the postmaster process, which commonly means they will appear on the console from which the cluster has been started.
- `syslog` means that the logs will be sent to an external syslog component.
- `csvlog` means that the logs will be produced as comma-separated values, useful for the automatic analysis of logs (more on this later).
- `jsonlog` means that the logs will be produced as JSON tuples, another format very useful for the automatic analysis of logs (more on this later).
- `eventlog` is a particular component available only on Microsoft Windows platforms that collects the logs of a whole bunch of services.

It is possible to set up the logging destination with multiple values, so that different destinations and types of logs will be produced.

Another important setting of the logging infrastructure is `log_collector`, which is a boolean value that fires on a process (named the logging collector) that captures all the logs sent to the standard error and stores them where you want. In short, setting `log_destination = stderr` will force PostgreSQL to send all log messages to the standard error, which is the console from which the service has been launched. Usually, there is no attached console since the daemon is launched in the background, and moreover, not many people want to keep a console open just to see the log messages scrolling on the screen. For this reason, `logging_collector = on` enables the PostgreSQL logging capture process, which aims to read all the messages produced on the standard error and send them to an appropriate destination. Usually, the destination would be a text file, a **Comma-Separated Values (CSV)** file, or something else. Therefore, `log_destination` decides where PostgreSQL will emit the log messages, while `logging_collector` fires a dedicated process to capture those emitted log and send them elsewhere. It is important to note that a few logging destinations also require the logging collector to be turned on: `cvslog` and `jsonlog` require the `logging_collector` to be enabled.

To summarize, the two preceding parameters are somewhat inter-dependent: you need to choose where to send the logs that PostgreSQL will always produce (`log_destination`), and in the case that you send them only (or also) to the standard error or to a custom format (like `cvslog`), you need to turn on a dedicated process (the `logging_collector` value) to catch any log entries and store them on disk. This means that your logging configuration will always be something like the following:

```
log_destination = 'stderr'
logging_collector = on
```

Here, the first line tells the cluster to send the produced logs to the standard error, but from there they are to be managed and stored by a dedicated process named the logging collector.

In the rest of this section, we will concentrate on the configuration of the logging collector. The logging collector can be configured to place logs in the directory you desire, to name the log files as you wish, and to automatically rotate them. Log rotation is a quite common feature in every logging system and means that once a single log file has grown to a specified size, or when enough time has passed, the file log is closed and a new one (with a different name) is created. For example, you can decide to automatically rotate your log files once a single file becomes 100 MB or every 2 days: the first condition that happens triggers the rotation so that PostgreSQL produces a different log file at least every 2 days or every 100 MB of textual information.



Log rotation is useful because it allows you to produce smaller log files that can be constrained to a specific period of time. On one hand, this is going to scatter the logs across multiple (possibly small) files; on the other hand it will not produce a single (possibly huge) log file that can be problematic to analyze.

Once you have enabled the logging collector, you have to configure it so that it will store the logs as you want and where you want. In fact, you can use the following parameters to configure the logging collector process, by placing the right value for any of the following settings in the PostgreSQL configuration file:

- `log_directory`: This is a directory where individual log files must be stored. It can be a relative path, considered with regard to `PGDATA`, or an absolute path (which the process must be able to write into). Clearly, it must be a path where the operating system user running the cluster has write access.
- `log_filename`: This is a single filename or a pattern to specify the name of every log file (within `log_directory`). The pattern can be specified following `strftime(3)` to format it with a date and time. For example, the value `postgresql-%Y-%m-%d.log` will produce a log filename with the date (respectively, year, month, and day), for example, `postgresql-2022-07-19.log`.
- `log_rotation_age`: This indicates how much time the log should wait before applying automatic log rotation. For example, `1d` means 1 day and specifies that the logs will be rotated once per day.
- `log_rotation_size`: This specifies the size of the log file before it is rotated to a new one. For example, `50MB` means that the log file will be rotated once it has reached a size of 50 MB.
- `log_truncate_on_rotation`: This boolean parameter determines whether PostgreSQL must truncate (i.e., empty and start over) an existing file when rotating, or instead append the new log data to the existing file.



Be aware that log rotation is not exact: the log files could slightly exceed the rotation size or age depending on the cluster logging activity.

All the rotation-related settings require the `logging_collector` to be turned on: after all, PostgreSQL can manage rotation only if it is in charge of the logging.